M.Tech Robotics Cornerstone Project

Reinforcement Learning-Based Control of a Collaborative Robot in Isaac Sim

UR10e + Robotiq 2F-85 | Isaac Sim + Isaac Lab | Scripted Control + PPO RL

Submitted by: Adarsh Pandey (2025JRB2021)
Pritosh Kumar (2025JRB2025)

Supervisor: Prof. Sunil Jha

Presentation flow: Motivation → Problem → Methodology → Results → Insights → Future Work

Motivation

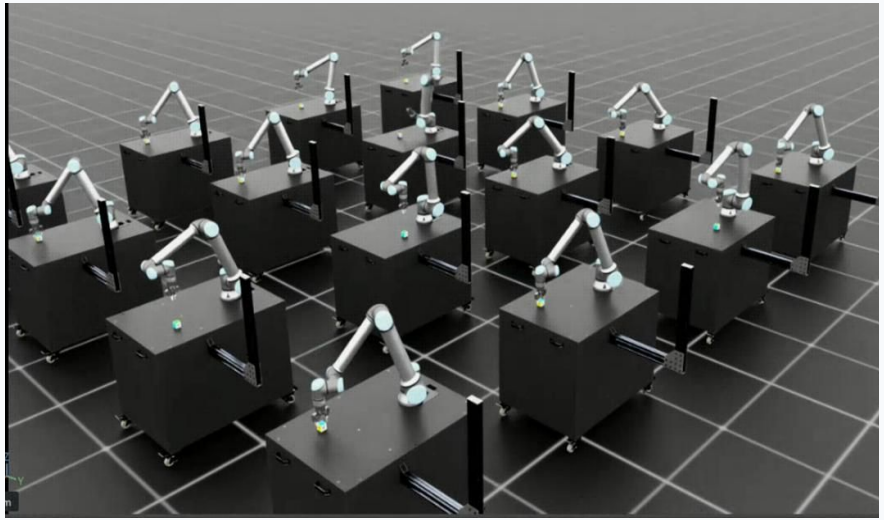
Why simulate and train robot manipulation before hardware deployment?

WHY ISAAC SIMULATION FIRST

Safe learning(Industrial perspective)
Policies can fail in simulation without damaging the real robot, gripper, or workspace.

Contact realism(Real world)
Cube interaction depends on friction, gravity, collision, TCP pose, and gripper timing.

Scalable experiments(Machine Learning)
Isaac Lab supports vectorized environments and RL training loops for rapid iteration.



Project motivation: ***Create a repeatable simulation-to-learning workflow for industrial pick-and-place, then use it as a bridge toward real robot execution.***

Problem Statement

Design a simulated UR10e cube pick-and-place task using scripted control and reinforcement learning.

SUCCESS CRITERIA

- TCP aligned to cube center
- Stable vertical descent
- Reliable grasp and lift
- Vectorized IsaacLab compatibility
- Clear reportable outputs

CORE TECHNICAL QUESTION

How can the robot reach the cube center with a correct vertical TCP pose, close at the correct instant, lift the cube, and repeat the task reliably?

Project Activity Flow and Gantt Plan

Fifteen-week work flow from environment setup to RL debugging and reporting.

S.No	Actvities	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Week 14	Week 15
1	Literature review of Isaac Sim and Isaac Lab															
		A/O														
2	Installation and environmental setup															
					A/O											
3	Custom project structure creation and scene reconstruction															
							A/O									
4	UR10e + Robotic Integration															
								A/O								
5	Manual control & debugging															
									A/O							
6	Scripted pick-and-place implementation															
											A/O					
7	RL environment creation															
												A/O				
8	Initial PPO training															
													A/O			
9	Action, Obsservation, Reward, termination setup															
														A/O		
10	Multi-robot vectorized demo testing															
															A/O	
11	Result Analysis and report preparation															
																A/O

	Plan
	Under Progress
	Actual

FLOW SUMMARY

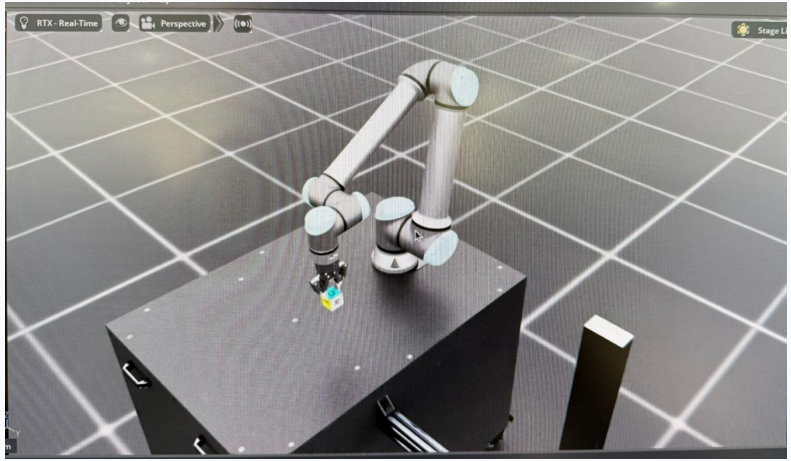
Study → Setup → Scene → Robot → Manual Debug → Scripted Pick-Place → RL Training → Debugging → Report

A/O = activity output achieved at that checkpoint. The plan keeps the scripted demo as completed and RL grasp/lift refinement as ongoing improvement.

The project flow matches the submitted reports: environment creation, robot integration, scripted manipulation, RL task creation, training/debugging, and reporting.

System Platform

Isaac Sim provides the physical world; Isaac Lab provides the learning framework.



ISAAC SIM

High-fidelity simulator for physics, contact, USD assets, rendering, collision, gripper–cube interaction, and visual inspection.

ISAAC LAB

Manager-based robot-learning layer for actions, observations, rewards, terminations, vectorized environments and PPO training.

Isaac Sim gives the physical tabletop manipulation world; Isaac Lab turns that world into a trainable RL task.

Simulation Environment Design

Physics + robot + cube + table implemented as a custom Isaac Lab task.

ENVIRONMENT IMPLEMENTATION

- Rigid cube with gravity and contact
- Solver iterations tuned for manipulation
- Episode timing: 0.01 s physics step, decimation 2
- Vectorized env count set from CLI

KEY DESIGN DECISION

The real control point is not the wrist body origin. A virtual TCP frame was defined using an offset from wrist_3_link and used consistently for IK, observations and rewards.

Robot Integration and Control Pipeline

UR10e motion is controlled through IK actions and a gripper command.

CONTROL MODES TESTED

- Manual control for joint/gripper debugging
- Scripted zero-agent with state machine phases
- IK absolute and IK relative action variants
- RSL-RL policy action: 6 arm + 1 gripper

WHY THIS PIPELINE MATTERED

RL was built only after manual and scripted control verified that robot motion, gripper action, cube physics and TCP geometry were meaningful.

Scripted Pick-and-Place Methodology

A deterministic controller was built first to solve the physical manipulation sequence.

KEY ALGORITHM IDEA

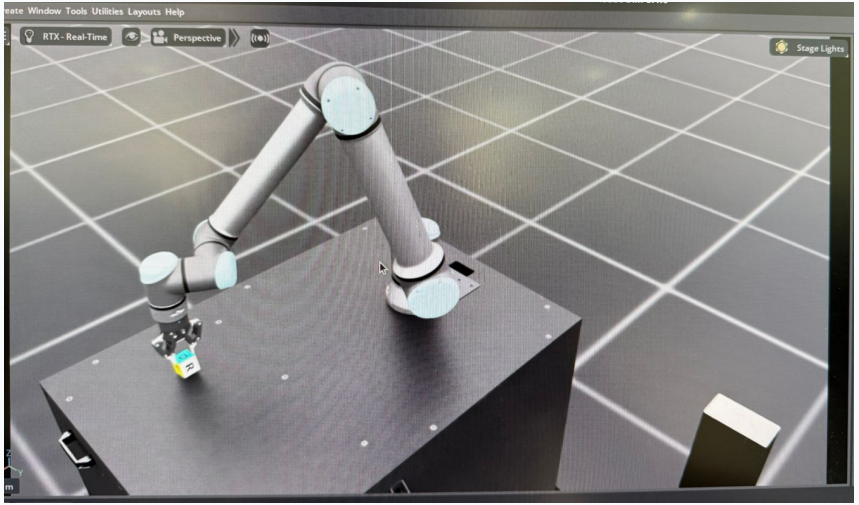
Use small delta-motion steps rather than jumping directly to the target. While descending, repeatedly correct XY alignment so the TCP remains above the cube center.

WHY IT WORKED

- Avoided side collisions with cube
- Maintained downward gripper orientation
- Delayed close command until grasp height
- Created a reference sequence for RL reward shaping

Critical Manipulation Insight: Wrist vs TCP

The gripper must align its grasp center, not merely the wrist body origin.



OBSERVED PROBLEM

The robot visually reached near the cube, but the wrist_3_link was aligned while the actual gripper center was offset. The cube was not reliably trapped between the fingers.

FIX IMPLEMENTED

Define virtual TCP/grasp center:

GRASP_CENTER_OFFSET = [0.0, 0.0, 0.18]

Result: rewards, observations and IK commands refer to the same physical grasp point.

Reinforcement Learning Formulation

The pick task was converted into a ManagerBasedRLEnv trained with PPO.

ACTION SPACE

- 6D arm command using differential IK
- 1D gripper command mapped to finger_joint

OBSERVATION SPACE

- Joint position and velocity
- Object position and target object pose
- TCP position and TCP-to-object vector
- Previous action vector

PPO CONFIGURATION

Actor/Critic MLP

256 → 256 → 128

γ = 0.99 (Discount factor)

Learning rate ≈ 3e-4

Reward Shaping Strategy

RL reward design evolved from sparse lift reward to scripted-style phase guidance.

Hover above object → XY align → Orient TCP → Vertical descent → Close at grasp → Lift + hold

INITIAL PROBLEM

A sparse lift reward stayed zero because the policy rarely discovered the full sequence: hover → align → descend → close → lift.

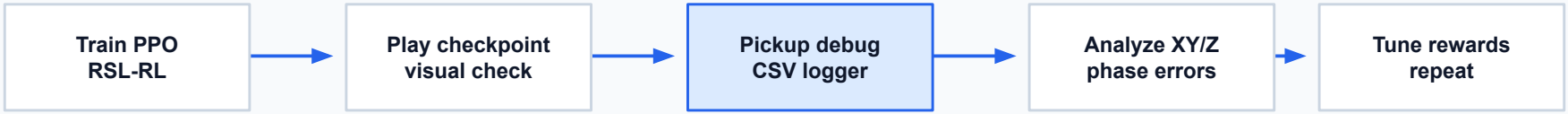
PENALTIES ADDED

- Low side approach penalty
- Close-before-vertical-grasp penalty
- Close-too-high penalty
- Action-rate smoothing

Design philosophy: teach the policy the same physical sequence that made the scripted controller successful.

Training and Debugging Pipeline

The project used repeatable commands and CSV logs to understand policy behavior.



IMPORTANT DEBUG SIGNALS

- Phase: DESCENDING_OPEN / AT_GRASP_OPEN
- TCP–cube XY error
- TCP height above cube
- Finger/gripper state
- Orientation error and action vector

KEY TRAINING INSIGHT

Numerical stability mattered. A joint-velocity penalty caused extreme negative reward/value loss in one run, so unstable penalties were removed or reduced during tuning.

Results and Demonstration Outputs

What the current project can show clearly and professionally.

1×

single robot pick-place demo

16×

vectorized multi-env demo target

7D

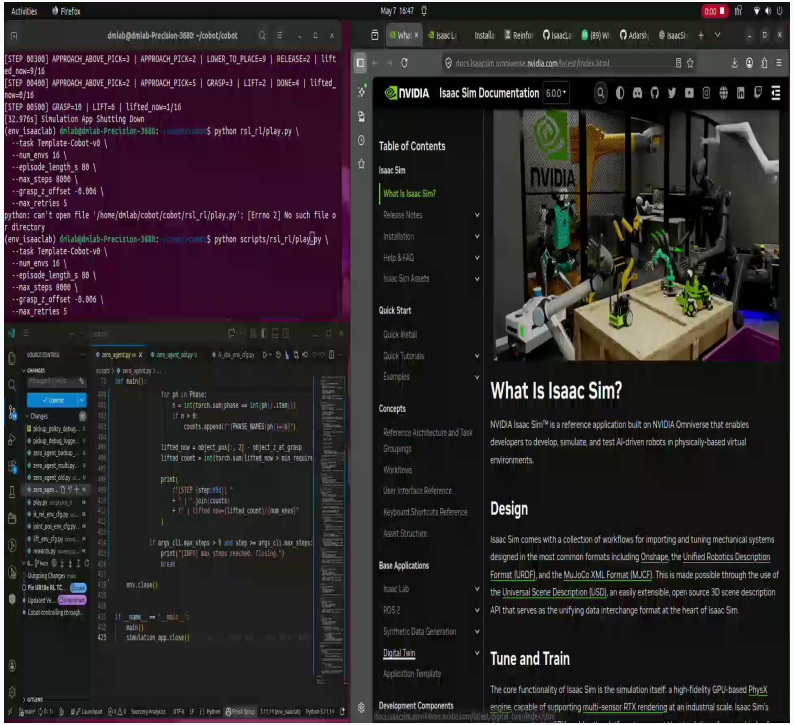
RL action space: 6 arm + 1 gripper

1000+

PPO iteration tests / debug runs

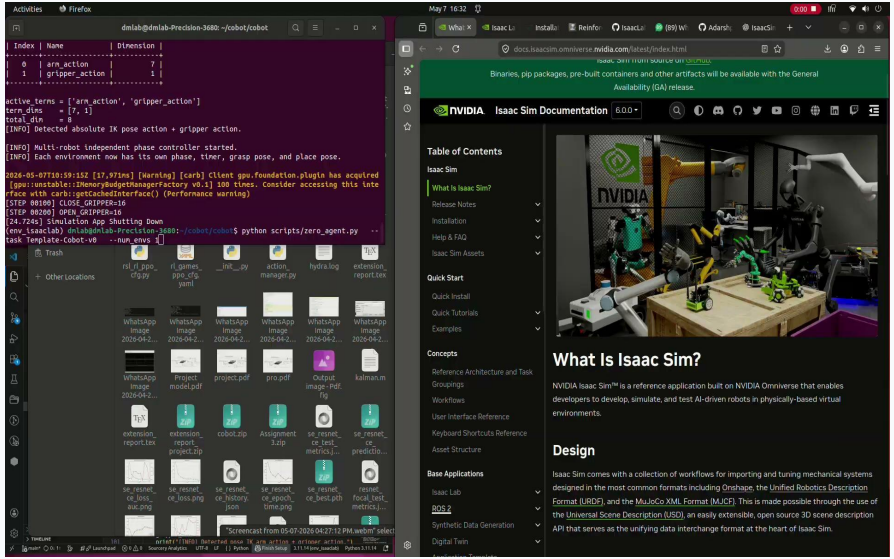
DEMO PLAN

1) **Single robot scripted pick-and-place**
2) **Multi-robot vectorized repeated pick-and-place**
3) **RL policy reaching/closing behavior with debug analysis**



Multi-robot vectorized repeated pick-and-place

Results and Demonstration Outputs



Simulation

Motion Implemented on Machine Lab - CMF

Results and Insights

A clear, honest summary of what worked and what remains open.

Status	Result Area	Evidence / Insight
Completed	**Environment + task registration**	Custom Isaac Lab environment loads and is reproducible.
Completed	**Manual control and debug tools**	Joint, gripper, cube, body names and TCP behavior verified.
Completed	**Scripted vertical pick-place**	Phase-based controller demonstrates the most reliable output.
Completed	**TCP-centered alignment**	Virtual TCP offset fixed wrist-vs-gripper-center mismatch.
Strong	**RL reaching + descent behavior**	Policy approaches the cube and improves alignment/closing behavior.
Partial	**RL reliable grasp + lift**	Grasp contact and lift reward still need tuning.

MAIN INSIGHT **Contact-rich manipulation is not solved by “reaching near the object.” Success depends on a consistent TCP frame, vertical approach, gripper timing and stable contact physics.**

Challenges Faced and Solutions

The main engineering problems and how they were addressed.

Challenge	Observed Effect	Solution
Setup and paths	Isaac/Conda/script paths caused launch issues	Validated envs and used repeatable CLI commands
Action mismatch	Scripts assumed wrong action dimensions	Inspected action manager and handled 7D / 8D cases
Wrong alignment point	Wrist aligned instead of gripper center	Virtual TCP offset + Frame Transformer
Side approach	Cube was pushed before grasp	Hover-above and vertical delta descent
Reward instability	Joint velocity term caused reward explosion	Removed/reduced unstable penalties
Multi-env variability	Only some robots picked reliably	Per-env phase logic and conservative demo settings

Novelty and Contributions

What makes the work valuable as a robotics simulation project.

1

Full custom Isaac Lab task

UR10e + Robotiq + cube/table physics implemented as a reproducible extension.

2

Scripted-to-RL methodology

Manual debug and scripted control used to inform RL reward design.

3

TCP-centered control insight

Wrist-vs-gripper-center issue identified and corrected with Frame Transformer offset.

4

Presentation-ready demos

Single robot and vectorized multi-env outputs organized for evaluation.

Contribution: a practical simulation framework connecting robotics, control, contact physics and reinforcement learning.

Future Work

Next steps to move from partial RL pickup toward robust manipulation.

Gripper contact tuning → **Curriculum learning** → **Imitation from scripted policy** → **Vision and sim-to-real**

Improve friction, stiffness, grasp height and close timing margins.

Train reach → descend → close → lift → place progressively.

Use successful scripted trajectories to bootstrap PPO behavior.

Connect camera, workspace calibration, and physical UR12e/Arduino gripper.

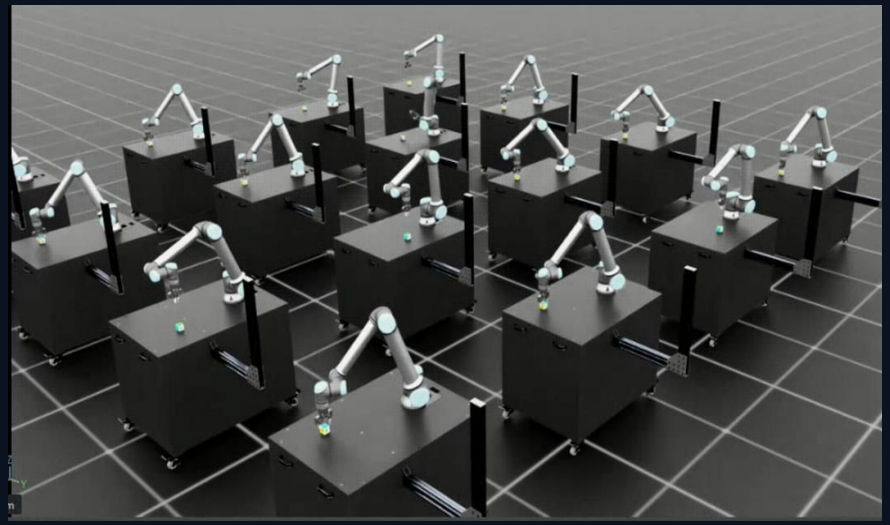
TARGET END STATE

A robust simulated and physical pick-and-place workflow using camera-based perception, calibrated workspace coordinates, validated gripper logic and a trained RL/imitative policy.

Conclusion

Built a complete UR10e Pick-and-place simulation and RL training pipeline.

Scripted manipulation is demonstrable; RL now has the correct environment, TCP frame, reward structure and debug tools for reliable grasp/ lift refinement.



Thank you!

Any Questions?